LAB2 part2 Оброблення даних про файли (робочий зошит)

Exported 2/28/2026, 2:28:42 PM

LAB2 Частина 2. Оброблення даних про файли

Виконавець: **Базилевич Олексій**, група **К-27**
Викладач: **Андрій Ляшко**

Нам знадобиться бібліотека `pandas`.
Бібліотека є сторонньою, отже потребує встановлення. Краще у віртуальне оточення. Мала бути встановлена, якщо ні (клітина з імпортуванням виконується з помилкою), то треба встановити. Можна прямо з Jupyter. Тільки не під час пари.

Наступну клітину можна розкоментовувати та виконувати тільки, якщо в якості ядра (kernel) вибране віртуальне оточення!

In [1]:

```
#!pip install pandas
```

Якщо встановлювати не в оточення, то хоча б тільки для користувача (з ключем --user).

In [2]:

```
#!pip3.14 install pandas --user    # Linux
```

In [3]:

```
#!py -3.14 -m pip install pandas --user    # Windows
```

The document was converted with **Code-Format**: https://code-format.com/

In [4]:

```
from pathlib import Path

import numpy as np
import pandas as pd
print(f'pandas {pd.__version__}')
```

```
pandas 3.0.1
```

Постановка задачі

Згадка про минуле (Лабораторну роботу №1)
Протягом навчання програмні проєкти з різних предметів зберігалися в папках з переліку `folders`, який оброблявся в лабораторній роботі №1.
Щоб профільтрувати допоміжні файли середовищ, було введено "білий список" розширень (`WHITE_LIST`) та список розширень файлів з текстом програм(задається) (`CODE_LIST`).
Також було написано функцію `kind`, що за розширенням повертає рядок з символом C,W або O відповідно для файлів з текстом програм (відповідно до переліку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список).

У результаті виконання лабораторної роботи для кожного місяця та кожного розширення, що зустрічаються при перегляді файлів, розміщених у папках зі списку, було знайдено таку інформацію:

`рік, місяць, розширення, кількість файлів, сумарний обсяг у байтах, кількість їх рядків.`

Кількість рядків наводиться тільки для розширень категорії C; білі та порожні рядки не рахуються; для інших розширень ця клітина таблиці залишається незаповненою.

За цією інформацією було побудовано csv-файл з іменем `lab1_1.csv` (кодування тільки utf-8), на початку якого розміщено рядок з заголовками: year, mon, ext, cnt, size, lines.

In [5]:

```
CODE_LIST = {'.cpp', '.hpp', '.c', '.h', '.py', '.pyw', '.cs', '.js',
'.java'}
WHITE_LIST = CODE_LIST | {'.html', '.json', '.txt', '.ipynb'}
```

In [6]:

```
headers = ['year', 'mon', 'ext', 'cnt', 'size', 'lines']
```

The document was converted with **Code-Format**: https://code-format.com/

За інформацією з csv-файлу з'ясувати

1. Скільки різних розширень зустрічається у папках з заданого списку, скільки файлів з кожним розширенням. (Для цієї задачі розглядаємо всі розширення, а не тільки "білого списку".)
   Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.
2. За датами створення файлів для кожного місяця, починаючи з вересня 2024-го року, знайти таку інформацію: кількість файлів з розширеннями з "білого списку", їх сумарний розмір у кілобайтах, кількість їх рядків (враховувати тільки для файлів категорії C), середній розмір у байтах, середню кількість рядків.
   Відповідну інформацію вивести на екран у табличному вигляді в порядку, оберненому до хронологічного.

Зверніть увагу на підписи стовпчиків, що будуть виводитись.

Всі значення округлити та привести до цілого типу.

Завантаження даних

Задаємо шлях до файлу `lab1_1.csv`.

In [7]:

```
path = Path('lab1_1.csv')
```

The document was converted with **Code-Format**: https://code-format.com/

Завантажимо дані за допомогою функції `pd.read_csv` ([документація тут](#)).

Зверніть увагу на те, що:

1. файл записано в кодуванні `utf-8`
2. перший рядок файлу містить заголовки year, mon, ext, cnt, size, lines
3. значення у стовпчику lines можуть бути відсутні

▶ Підказки.
Функції варто передати параметри, що задають:

1. кодування, в якому записано текстовий файл
2. номер рядка, в якому записано заголовки
3. тип, в якому слід записати в пам'ять останній стовпчик

▶ Ще більше підказок

1. параметр sep задає роздільники полів
2. параметр encoding задає кодування
3. параметр header задає номер рядка, в якому записано заголовки (нумерація починається з 0)
4. параметр dtype може бути словником, що задає типи стовпчиків (можливо, не всіх); назві стовпчика відповідає назва типу
5. тип Int64 містить цілі числа та особливе значення NA

In [8]:

```
data = pd.read_csv(path, encoding='utf8', sep=",", header=0,
dtype={"lines": "Int64"}, names=headers)
data.head(3)
```

	year	mon	ext	cnt	size	lines
0	2021	12	NaN	2	3422	<NA>
1	2021	12	.bas	2	7979	<NA>
2	2021	12	.bat	294	85220	<NA>

The document was converted with **Code-Format**: https://code-format.com/

In [9]:

```
data.tail(3)
```

	year	mon	ext	cnt	size	lines
1021	2026	2	.xlsx	14	293131	<NA>
1022	2026	2	.yaml	18	2025769	<NA>
1023	2026	2	.zip	4	2240635049	<NA>

Подивимось інформацію про завантажені дані. Документація на метод `info`.

In [10]:

```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1024 entries, 0 to 1023
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   year    1024 non-null   int64
 1   mon     1024 non-null   int64
 2   ext     989 non-null    str
 3   cnt     1024 non-null   int64
 4   size    1024 non-null   int64
 5   lines   94 non-null     Int64
dtypes: Int64(1), int64(4), str(1)
memory usage: 49.1 KB
```

The document was converted with **Code-Format**: https://code-format.com/

Бачимо, що відсутні значення є не тільки в стовпчику `lines`, що природньо, бо кількість рядків обчислювалась не для всіх розширень, але і в стовпчику `ext` з розширеннями.

Замінимо відсутні розширення порожнім рядком.

Використаємо метод `fillna`.

Слід замінити тільки відсутні розширення, тому значення параметра `value` слід зробити словником, що для кожного стовпчика, який слід оброблювати, задає значення-заповнювач.
Не забудьте, що перетворення слід виконувати "на місці".

▸ Підказка (тільки за повного безвихіддя)

```
data.fillna(value={'ext':''}, inplace=True)
```

Альтернатива. Можна спробувати перетворити тільки стовпчик `data.ext`.

▸ Підказка (тільки за повного безвихіддя)

Використаємо метод `fillna` класу `Series`, щоб виконати заміну, і присвоїмо результат стовпчику `data.ext`. `data.ext = data.ext.fillna(value='')`

Перший спосіб більш економний за пам'яттю. Другий спосіб корисний тоді, коли для різних стовпчиків використовують різні алгоритми заповнення відсутніх значень.

In [11]:

```
data.fillna(value={'ext':''}, inplace=True)
data.head(3)
```

	year	mon	ext	cnt	size	lines
0	2021	12		2	3422	<NA>
1	2021	12	.bas	2	7979	<NA>
2	2021	12	.bat	294	85220	<NA>

The document was converted with **Code-Format**: https://code-format.com/

In [12]:

```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1024 entries, 0 to 1023
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   year    1024 non-null   int64
 1   mon     1024 non-null   int64
 2   ext     1024 non-null   str
 3   cnt     1024 non-null   int64
 4   size    1024 non-null   int64
 5   lines   94 non-null     Int64
dtypes: Int64(1), int64(4), str(1)
memory usage: 49.1 KB
```

Тепер незаповнені дані залишились тільки у стовпчику `lines`.

`Ні в якому разі там не повинно бути нулів!`

Групування та групові операції

In [13]:

```
data.head()
```

	year	mon	ext	cnt	size	lines
0	2021	12		2	3422	<NA>
1	2021	12	.bas	2	7979	<NA>
2	2021	12	.bat	294	85220	<NA>
3	2021	12	.c	1	4402	141
4	2021	12	.cbp	105	172765	<NA>

The document was converted with **Code-Format**: https://code-format.com/

Задача 1 Скільки різних розширень зустрічається в папках з заданого списку, скільки файлів з кожним розширенням?

Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

Скільки різних розширень зустрічається в папках з заданого списку?

Найпростіше: взяти стовпчик таблиці, що зберігає розширення, та знайти кількість унікальних значень.

▸ Підказки

1. Для отримання стовпчика таблиці `t` з іменем `name` можна використати вираз `t.name` або `t['name']`
2. Кількість унікальних елементів ряду можна знайти за допомогою методу `nunique` ([докуемнтаці тут](#)).

In [14]:

```
data.ext.nunique()
```

```
284
```

Скільки файлів з кожним розширенням?

Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

The document was converted with **Code-Format**: https://code-format.com/

Спочатку треба підсумувати інформацію по кожному розширенню. Допоможе метод `groupby` [документація тут](#).

Групувати слід за розширенням (параметр `by`) і для кожної групи обчислити суму кількостей файлів (метод `sum`).

Щоб не сумувати роки, місяці тощо, групування слід застосувати до вибірки потрібних стовпчиків. Маємо вибрати стовпчик, за яким групуємо, і стовпчик, який сумуємо.

▸ Підказка
Щоб вибрати потрібні стовпчики, можна використати операцію індексації по сукупності стовпчиків, наприклад, `dataFrame[['columnA','columnB']]`

Результат присвоїти змінній `task1`.

In [15]:

```
task1  = data[['ext', 'cnt']].groupby(by='ext').sum()
task1
```

	cnt
ext	
	77311
.0	17
.1	4
.14	2
.17	2
...	...
.xrc	2
.yaml	76
.yml	142
.zi	2
.zip	17

284 rows × 1 columns

The document was converted with **Code-Format**: https://code-format.com/

Відсортувати за спаданням кількості файлів. Допоможе метод `sort_values` (документація тут).

Не забудьте вказати, що сортування відбувається за спаданням та "на місці".

▸ Підказки

1. параметр `by` може задавати по яких стовпчиках сортувати
2. параметр `ascending` задає спадання-зростання
3. параметр `inplace` задає чи виконується перетворення на місці або створюється копія

In [16]:

```
task1.sort_values(by='cnt', ascending=False, inplace=True)
task1.head(3)
```

	cnt
ext	
.meta	111220
.png	84997
	77311

За умовою ще треба додати стовпчик з ознакою категорії розширення файлу W,C,O (див. умову). У лабораторній роботі №1 була написана функція `kind`, що за розширенням повертає рядок з шуканою ознакою. Використаємо її.

In [17]:

```
def kind(extension):
    if extension in WHITE_LIST:
        return 'W'
    elif extension in CODE_LIST:
        return 'C'
    else:
        return 'O'
```

Додамо до таблиці стовпчик з ознаками розширень файлів.

Зараз розширення є індексом. За допомогою методу метод `reset_index` (документація тут) зробимо його стовпчиком таблиці.

Не забувайте виконувати перетворення "на місці".

The document was converted with **Code-Format**: https://code-format.com/

In [18]:

```
task1.reset_index(inplace=True)
task1.head(3)
```

	ext	cnt
0	.meta	111220
1	.png	84997
2		77311

Тепер вже додаємо стовпчик з ознаками.

▶ Підказки

1. Щоб додати до таблиці `t` стовпчик `name`, можна виконати присвоювання виразу `t['name']`
2. Присвоювати можна скаляр (не наш випадок) або інший стовпчик.
3. Щоб отримати стовпчик з ознаками, можна застосувати метод `apply` ([документація тут](#)) до стовпчика з розширеннями.

In [19]:

```
task1['kind']=task1['ext'].apply(kind)
task1.head(3)
```

	ext	cnt	kind
0	.meta	111220	O
1	.png	84997	O
2		77311	O

За допомогою методу `set_index` (документація тут) знов зробимо розширення індексом таблиці.

Тут: Значенням параметра `keys` має стати ім'я стовпчика, що слід використати як індекс.

Не забувайте виконувати перетворення "на місці".

The document was converted with **Code-Format**: https://code-format.com/

In [20]:

```
task1.set_index(keys='ext', inplace=True)
task1.head(3)
```

	cnt	kind
ext		
.meta	111220	O
.png	84997	O
	77311	O

Змінимо написи.

In [21]:

```
task1.index.name='Розширення'
task1.columns=['Кількість файлів','Ознака']
task1.head()
```

	Кількість файлів	Ознака
Розширення		
.meta	111220	O
.png	84997	O
	77311	O
.cs	70314	W
.py	47833	W

The document was converted with **Code-Format**: https://code-format.com/

In [22]:

```
task1.head(20)
```

	Кількість файлів	Ознака
Розширення		
.meta	111220	O
.png	84997	O
	77311	O
.cs	70314	W
.py	47833	W
.pyc	16264	O
.txt	11587	W
.md	10759	O
.tex	6362	O
.pyi	4451	O
.json	3220	W
.mvfrm	2997	O
.dll	2950	O
.bin	2620	O
.dat	2192	O
.js	1696	W
.pdb	1631	O
.rsp	1542	O
.asmdef	1508	O
.mat	1326	O

The document was converted with **Code-Format**: https://code-format.com/

Задача 2 За датами створення файлів для кожного місяця знайти таку інформацію: кількість файлів з розширеннями з "білого списку", їх сумарний розмір у кілобайтах, кількість їх рядків (враховувати тільки для файлів категорії C), середній розмір у байтах, середню кількість рядків.

Відповідну інформацію вивести на екран у табличному вигляді в порядку, оберненому до хронологічного.

Всі значення округлити та привести до цілого типу.

План:

1. Додати до початкової таблиці `data` стовпчик `kind`, що містить ознаки розширень.
2. Вибрати дані, що стосуються розширень з "білого списку".
3. Згрупувати вибрані дані в межах року та місяця.
4. За згрупованою інформацією обчислити потрібні агрегати.
5. Виконати перетворення стовпчиків та відсортувати.

Щоб за невдачі повторно не завантажувати дані, створимо їх копію.

In [23]:

```
data1 = data.copy()
```

1. Додати до початкової таблиці `data` стовпчик `kind`, що містить ознаки розширень.

Аналогічно тому, що виконувалось раніше, додати до таблиці `data1` стовпчик `kind`, що містить ознаки.

In [24]:

```
data1['kind']=data1.ext.apply(kind)
data1.head(3)
```

	year	mon	ext	cnt	size	lines	kind
0	2021	12		2	3422	<NA>	O
1	2021	12	.bas	2	7979	<NA>	O
2	2021	12	.bat	294	85220	<NA>	O

The document was converted with **Code-Format**: https://code-format.com/

2. Вибрати дані, що стосуються розширень з "білого списку".

Запишіть вираз, що задає вибірку тільки інформації про розширення з "білого списку". Щоб далі не копіювати сам вираз, присвойте його значення змінній `selection`. (Це є зручно, коли експериментуємо, у продакшн потреби в цій змінній насправді не буде, замість неї можна буде використовувати вираз безпосередньо.)

▸ Підказка

- Розширення знаходиться в білому списку, якщо його ознакою є не O.
- Обчислити умову, що розширення знаходиться в білому списку, і використати її як аргумент операції індексації.

In [25]:

```
selection = data1[(data1.kind != 'O') & ((data1.year > 2024) |
((data1.year == 2024) & (data1.mon >= 9)))]
selection.head()
```

	year	mon	ext	cnt	size	lines	kind
288	2024	9	.c	1	27620	750	W
291	2024	9	.cpp	7	92626	1918	W
294	2024	9	.h	4	1160	57	W
295	2024	9	.json	5	2307	<NA>	W
301	2024	9	.txt	3	1827	<NA>	W

3. Згрупувати вибрані дані за ознаками в межах року та місяця.

   Допоможе метод `groupby` [документація тут](#).

Групувати слід за роком та місяцем (параметр `by` має бути списком). Оскільки далі будемо обчислювати статистики для розміру та кількості рядків, то перед групуванням з таблиці слід вибрати стовпчики, за якими групуємо і за якими обчислюємо статистики.

In [26]:

```
task2 = selection[['year', 'mon', 'cnt', 'size',
'lines']].groupby(by=['year','mon'])
```

The document was converted with **Code-Format**: https://code-format.com/

Далі необхідно знайти кількість файлів за кожен місяць, їх сумарний та середній розмір, а також сумарну та середню кількість рядків.

У результаті групування отримався об'єкт `DataFrameGroupBy` (документація тут).

До нього застосовний метод `agg`, що обчислює статистики (агрегатні характеристики) (документація тут).

Для стовпчика `cnt` будемо шукати суму значень (`sum`), для стовпчика `size` будемо шукати суму значень та середнє значення (`mean`), для стовпчика `lines` будемо шукати суму значень та середнє значення.

Першим параметром методу `agg` зробимо словник, що задає які статистики для яких стовпчиків слід обчислювати. Ключами словника мають бути імена стовпчиків (рядки), а значеннями списки статистик або окремі статистики (статистики також задаємо рядками-іменами)).

Результат агрегування зробимо значенням змінної `res2`.

In [27]:

```
res2 = task2.agg({'cnt':'sum', 'size':['sum', 'mean'], 'lines':
['sum', 'mean']})
res2.head(3)
res2.head()
```

		cnt	size		lines	
		sum	sum	mean	sum	mean
year	**mon**					
2024	**9**	20	125540	25108.000000	2725	908.333333
	10	1864	5440894	777270.571429	50299	12574.75
	11	610	747916	186979.000000	13819	4606.333333
	12	3	8167	4083.500000	211	105.5
2025	**1**	10	67140	13428.000000	1801	600.333333

Оскільки відбувалось групування по кільком стовпчикам та для окремих стовпчиків обчислювалось кілька характеристик, то отримали складені індекси (multindex) і рядків, і стовпчиків.

Поточний індекс стовпчиків.

The document was converted with **Code-Format**: https://code-format.com/

In [28]:

```
res2.columns
```

```
MultiIndex([(  'cnt',  'sum'),
            ( 'size',  'sum'),
            ( 'size', 'mean'),
            ('lines',  'sum'),
            ('lines', 'mean')],
           )
```

Перейдемо до простого індексу стовпчиків.

In [29]:

```
res2.columns = res2.columns.to_flat_index()
res2.columns
```

```
Index([   ('cnt', 'sum'),   ('size', 'sum'),  ('size', 'mean'),
        ('lines', 'sum'), ('lines', 'mean')],
      dtype='object')
```

In [30]:

```
res2.head(3)
```

		(cnt, sum)	(size, sum)	(size, mean)	(lines, sum)	(lines, mean)
year	**mon**					
2024	**9**	20	125540	25108.000000	2725	908.333333
	10	1864	5440894	777270.571429	50299	12574.75
	11	610	747916	186979.000000	13819	4606.333333

Змінимо назви стовпчиків.

The document was converted with **Code-Format**: https://code-format.com/

In [31]:

```
res2.columns=['Кількість', 'Сумарний розмір', 'Середній розмір',
'Кількість рядків', 'Середня кількість рядків',]
res2.head(3)
```

		Кількість	Сумарний розмір	Середній розмір	Кількість рядків	Середня кількість рядків
year	**mon**					
2024	**9**	20	125540	25108.000000	2725	908.333333
	10	1864	5440894	777270.571429	50299	12574.75
	11	610	747916	186979.000000	13819	4606.33333

Виразимо сумарний розмір у kb.

Для цього слід поділити значення на 1024, округлити. Додатково перетворимо на цілий тип (метод `astype` дозволяє виконувати перетворення типів, [документація тут](#)). Також не забудемо змінити підпис стовпчика.

Переходимо до кілобайтів.

In [32]:

```
res2['Сумарний розмір'] = (res2['Сумарний
розмір']/1024).round().astype(int)
```

Змінюємо підпис стовпчиків

In [33]:

```
res2.columns=['Кількість', 'Сумарний розмір, kb', 'Середній розмір,
b', 'Кількість рядків', 'Середня кількість рядків',]
```

The document was converted with **Code-Format**: https://code-format.com/

In [34]:

```
res2.head(3)
```

		Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
year	**mon**					
2024	**9**	20	123	25108.000000	2725	908.333333
	10	1864	5313	777270.571429	50299	12574.75
	11	610	730	186979.000000	13819	4606.33333

Округлити середній розмір та середню кількість рядків, перейти до цілих значень.

In [35]:

```
res2.info()
```

```
<class 'pandas.DataFrame'>
MultiIndex: 18 entries, (np.int64(2024), np.int64(9)) to
(np.int64(2026), np.int64(2))
Data columns (total 5 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Кількість                18 non-null     int64
 1   Сумарний розмір, kb      18 non-null     int64
 2   Середній розмір, b       18 non-null     float64
 3   Кількість рядків         18 non-null     Int64
 4   Середня кількість рядків 17 non-null     Float64
dtypes: Float64(1), Int64(1), float64(1), int64(2)
memory usage: 1001.0 bytes
```

The document was converted with **Code-Format**: https://code-format.com/

Ось тільки тепер замінимо відсутні значення кількості рядків нулями!

Якщо б це було зроблено раніше, то середня кількість рядків була б обрахована неправильно (на неї вплинули б нульові значення замість відсутніх).

▸ Підказка

Використати метод `fillna` (документація тут). Оскільки відсутні дані тільки в одному стовпчику, то все просто. Тільки не забудьте, що треба перетворювати наявний датасет, а не створювати новий.

In [36]:

```
res2.fillna(0, inplace=True)
res2.info()
```

```
<class 'pandas.DataFrame'>
MultiIndex: 18 entries, (np.int64(2024), np.int64(9)) to
(np.int64(2026), np.int64(2))
Data columns (total 5 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Кількість                 18 non-null     int64
 1   Сумарний розмір, kb       18 non-null     int64
 2   Середній розмір, b        18 non-null     float64
 3   Кількість рядків          18 non-null     Int64
 4   Середня кількість рядків  18 non-null     Float64
dtypes: Float64(1), Int64(1), float64(1), int64(2)
memory usage: 1001.0 bytes
```

Тепер можна перетворювати типи стовпчиків.

In [37]:

```
fname = 'Середній розмір, b'
res2[fname] = res2[fname].round().astype(int)

fname = 'Середня кількість рядків'
res2[fname] = res2[fname].round().astype(int)
```

The document was converted with **Code-Format**: https://code-format.com/

In [38]:

```
res2
```

		Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
year	**mon**					
2024	**9**	20	123	25108	2725	908
	10	1864	5313	777271	50299	12575
	11	610	730	186979	13819	4606
	12	3	8	4084	211	106
2025	**1**	10	66	13428	1801	600
	2	82	488	83310	11065	3688
	3	35387	345468	58959871	4707841	1569280
	4	33455	280612	57469303	4403387	1467796
	5	411	9962	1275176	27228	5446
	6	41	3841	1310986	2019	1010
	7	16	122	31157	3457	1152
	8	5	2	2437	0	0
	9	219	1256	257303	21947	7316
	10	429	2525	517021	39730	13243
	11	1224	4661	795394	31862	7966
	12	151	3781	774301	77363	38682
2026	**1**	20000	452666	42139076	7662090	1094584
	2	19979	80508	10304997	294211	73553

Відсортуємо в порядку, оберненому до хронологічного.

У даному випадку слід сортувати індекс. Допоможе метод `sort_index` ([документація тут](#)).

The document was converted with **Code-Format**: https://code-format.com/

In [39]:

```
res2.sort_index(ascending=[False, False], inplace=True)
res2
```

		Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
year	**mon**					
2026	**2**	19979	80508	10304997	294211	73553
	1	20000	452666	42139076	7662090	1094584
2025	**12**	151	3781	774301	77363	38682
	11	1224	4661	795394	31862	7966
	10	429	2525	517021	39730	13243
	9	219	1256	257303	21947	7316
	8	5	2	2437	0	0
	7	16	122	31157	3457	1152
	6	41	3841	1310986	2019	1010
	5	411	9962	1275176	27228	5446
	4	33455	280612	57469303	4403387	1467796
	3	35387	345468	58959871	4707841	1569280
	2	82	488	83310	11065	3688
	1	10	66	13428	1801	600
2024	**12**	3	8	4084	211	106
	11	610	730	186979	13819	4606
	10	1864	5313	777271	50299	12575
	9	20	123	25108	2725	908

Частину 2 зроблено!

The document was converted with **Code-Format**: https://code-format.com/

The document was converted with **Code-Format**: https://code-format.com/